

Session 16: Analysis of Omics Big Data using Machine Learning Techniques, Part II.

Hokeun Sun

Professor
Department of Statistics
Pusan National University

① Lasso for Variable Selection

- Linear regression
- Lasso regularization
- Cross-validation
- R practice: Heart data

② Elastic-net for Gene Selection

- Penalized regression
- Elastic-net regularization
- R practice: Breast cancer gene expression data

③ Group Lasso for Genetic Pathway Selection

- Group lasso
- Overlapping group lasso
- Bi-level selection methods
- R practice: Breast cancer gene expression data

④ Analysis of High-dimensional DNA Methylation Data

- Introduction to DNA methylation data
- Ovarian cancer study
- R practice: Ovarian cancer DNA methylation data

1. Lasso for Variable Selection

- 2. Elastic-net for Gene Selection
- 3. Group Lasso for Genetic Pathway Selection
- 4. Analysis of High-dimensional DNA Methylation Data

Statistical Modeling

- **Statistical learning** is a set of tools for modeling and understanding complex datasets.
- Outcome measurement Y (also called dependent variable, response, phenotype outcome).
- Vector of p predictor measurements X (also called inputs, regressors).
 - Genomic features (SNP, gene expression levels, ...)
 - Clinical/environmental features (age, gender, ...)
- In the **regression problem**, Y is quantitative (e.g numerical values of trait, blood pressure, cholesterol levels, ...)
- In the **classification problem**, Y takes values in a finite, unordered set (e.g., survived/died, cancer class of tissue sample, case/control, ...)

- Now we write our model as

$$Y = f(X) + \epsilon$$

where ϵ captures measurement **errors** and other discrepancies.

- With a good $f(\cdot)$ we can make predictions of Y at new points $X = x$.
- Depending on the complexity of $f(\cdot)$, we may be able to understand how each component X_j of X affects Y .
- The function f that connects the input variable to the output variable is in general **unknown**.
- There are relatively many approaches for **estimating** the function f .

Linear Regression

- If Y is quantitative, the **linear regression** is defined as

$$\begin{aligned} Y &= f(X_1, \dots, X_p) + \epsilon \\ &= \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \epsilon, \end{aligned}$$

where $X = (X_1, X_2, \dots, X_p)$.

- Linear regression assumes that the dependence of Y on $X_1, X_2, \dots, X_p$ is **linear**.
- True regression functions are **never linear**!
- Although it may seem overly simplistic, linear regression is extremely useful both conceptually and practically.
- The linear model has distinct advantages in terms of its **interpretability** and often shows good **predictive performance**.

Least Square Estimate

- In linear regression, regression coefficients,

$$\beta = \{\beta_0, \dots, \beta_p\}$$

are unknown and need to be estimated.

- The **least square (LS)** estimate for β minimizes the residual sum of squares. i.e.,

$$\hat{\beta}^{LS} = \arg \min_{\beta \in \mathbb{R}^{p+1}} \text{RSS}(\beta)$$

where

$$\text{RSS}(\beta) = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2$$

Limitation of Least Square Estimate

- **Gauss-Markov Theorem:** The LS estimates for β have the **smallest variance** among all linear **unbiased** estimates.
- Problems in multiple linear regression
 - Some coefficients in β are not well estimated and have large variance for **correlated predictors**, since correlated variables carry the same information regarding the response.
 - **LS** cannot be computed when $n < p$ (high-dimensional data) due to **singularity**.
- As an alternative, we can fit a model containing all p predictors using a technique that **constrains** or **regularizes** the coefficient estimates, or equivalently, that **shrinks** the coefficient estimates towards **zero**.
 - Ridge regression
 - Lasso (**L**east **a**bsolute **s**hrinkage and **s**election **o**perator).

Lasso Regularization

- Lasso coefficient estimates solve the problems

$$\underset{\beta}{\text{minimize}} \sum_i^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \quad \text{subject to} \quad \sum_{j=1}^p |\beta_j| \leq s$$

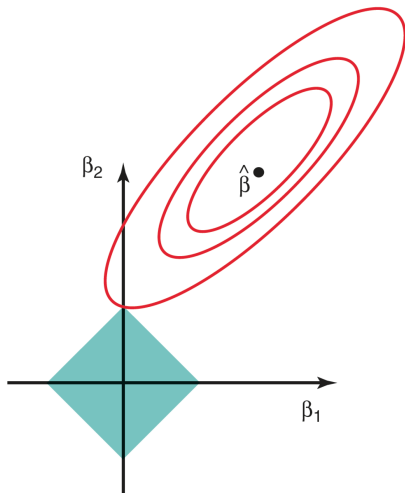
- The solution is equivalent to minimize the quantity

$$\text{RSS}(\beta) + \lambda \|\beta\|_1 = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p |\beta_j|,$$

where $\lambda > 0$ is a tuning parameter to control sparsity.

The Geometric View of Lasso

- When $p = 2$, Lasso solution is



Lasso Regularization

- The l_1 penalty of the **lasso** has the effect of forcing some of the coefficient estimates to be exactly equal to zero when the tuning parameter λ is sufficiently large.
- Hence, the lasso performs **variable selection**.
- The lasso yields **sparse** models - that is, models that involve only a subset of the variables.
- The tuning parameter λ controls the number of **nonzero** regression coefficients (**degrees of freedom**).
 - If λ increases, the degrees of freedom decrease.
 - If λ decreases, the degrees of freedom increase.
- Selecting an appropriate value of λ for the lasso is **crucial**.
- In general, the optimal λ value is chosen among a grid of λ values

Lasso Regularization

- For a grid of λ values such as

$$\lambda_{\max} = \lambda_1 > \lambda_2 > \dots > \lambda_{m-1} > \lambda_m = \lambda_{\min},$$

the number of **nonzero regression coefficients** (degrees of freedom: df) is

$$df(\hat{\beta}_{\lambda_1}) = 0 \leq df(\hat{\beta}_{\lambda_2}) \leq \dots \leq df(\hat{\beta}_{\lambda_{m-1}}) \leq df(\hat{\beta}_{\lambda_m})$$

- The **optimal value** of λ should be chosen by a resampling method such as **cross-validation**.
- Numerical values of **nonzero regression coefficients** also rely on λ .

Training Set

- In general, two types of data are considered such as the **training set** and the **test set**.
- The main goal is to minimize the **test error**.
- **Training set** is used to train the model, i.e., to estimate the regression coefficients. Suppose that we have m different models

$$\hat{f}_{\lambda_1}(x), \hat{f}_{\lambda_2}(x), \hat{f}_{\lambda_3}(x), \dots, \hat{f}_{\lambda_m}(x)$$

- The l -th model has $\hat{\beta}_l = \{\hat{\beta}_{0,l}, \hat{\beta}_{1,l}, \dots, \hat{\beta}_{p,l}\}$ such that

$$\hat{f}_{\lambda_l}(x) = \hat{\beta}_{0,l} + \hat{\beta}_{1,l}x_1 + \dots + \hat{\beta}_{p,l}x_p$$

- $\hat{\beta}_l$ should be estimated based only on the **training set**.

Test Set

- **Test set** is used to assess the model performance. The **test set** data should not be used in the model building process.
- Given the **test** data $T = \{x_i, y_i\}$ for $i \in \{1, \dots, N\}$, the **mean squared error (MSE)** of λ_l is

$$MSE_{\lambda_l} = \frac{1}{N} \sum_{i \in T} \left(y_i - \hat{f}_{\lambda_l}(x_i) \right)^2$$

- We can find the **best model** among m models by comparing the **test error** of m models.

$$\hat{\lambda} = \arg \min_{\lambda_1, \dots, \lambda_m} MSE_{\lambda_l}$$

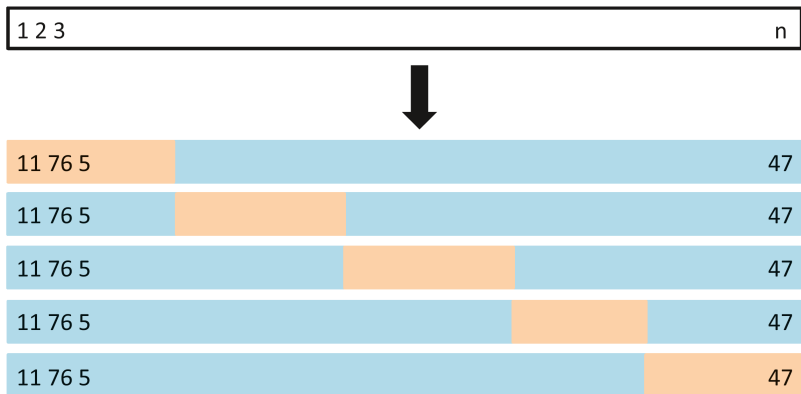
- **Test set** is often called **validation set** in tuning parameter selection.

K -fold Cross-validation

- In order to compute the **test error**, we need to have a large designated test set. However, it is not often available.
- The **K -fold cross-validation (CV)** is the most popular approach to estimate the **test error**.
- For each model,
 - ① Randomly divide the data into K equal-sized parts.
 - ② Leave out part k , and estimate the regression coefficients based on the other $K - 1$ parts (**training set**).
 - ③ Compute MSE for the left-out k th part (**test set**).
 - ④ This is done in turn for each part $k = 1, 2, \dots, K$.
 - ⑤ Final MSE value is then averaged over K parts.

K -fold Cross-validation

- Divide data into K roughly equal-sized parts ($K = 5$ here)



K -fold CV for Lasso

- Suppose that y_i is a **quantitative** value of the i -th individual.
- For lasso, the K -fold CV procedure:
 - ① Randomly separate n samples into K folds: $C_1, C_2, \dots, C_K$
 - ② For each λ , compute regression coefficients based on C_{-k} ,

$$\hat{\beta}_{\lambda_1}^{[-k]}, \hat{\beta}_{\lambda_2}^{[-k]}, \dots, \hat{\beta}_{\lambda_{m-1}}^{[-k]}, \hat{\beta}_{\lambda_m}^{[-k]}$$

Note that C_{-k} is an observation set with part k removed.

- ③ Compute **cross-validation error (CVE)** for each λ .

$$\text{CVE}(\lambda_l) = \frac{1}{n} \sum_{k=1}^K \sum_{i \in C_k} \left(y_i - x_i^T \hat{\beta}_{\lambda_l}^{[-k]} \right)^2$$

for $l = 1, 2, \dots, m$.

- ④ Pick up the optimal $\hat{\lambda}_l$ that minimizes $\text{CVE}(\hat{\lambda}_l)$.

CV Error for Binary Outcomes

- Suppose that $y_i = 1$ for a **case** and $y_i = 0$ for a **control**.
- **CVE** based on **deviance**

$$\text{CVE} = \frac{1}{n} \sum_{k=1}^K -2 \sum_{i \in C_k} \left(y_i \log \hat{p}_i^{[-k]} + (1 - y_i) \log(1 - \hat{p}_i^{[-k]}) \right),$$

where $\hat{p}_i^{[-k]}$ is the probability of $y_i = 1$ for the observation i , obtained from the data with part k removed.

- **CVE** based on **classification error**

$$\text{CVE} = \frac{1}{n} \sum_{k=1}^K \sum_{i \in C_k} I \left(y_i - \hat{y}_i^{[-k]} \right),$$

where $\hat{y}_i^{[-k]}$ is the fit for the observation i , obtained from the data with part k removed.

Example: Heart Data

- These data contains a binary outcome **AHD** for 303 patients who presented with chest pain.
- An outcome value of **Yes** indicates the presence of heart disease based on an angiographic test, while **No** means no heart disease.
- There are 13 predictors including **Age**, **Sex**, **Chol** (a cholesterol measurement), and other heart and lung function measurements.
- We can find some **missing values** from data, so we removed the samples with **missing values**. Then, the sample size is reduced down to 297.

Example: Heart Data

```
library(devtools)
install_github("statsun78/kogo")
library(kogo)
```

```
data(Heart)
str(Heart)
head(Heart)
summary(Heart)
```

```
g0 <- glm(AHD ~., family="binomial", Heart)
summary(g0)
```

```
x <- model.matrix(AHD ~., Heart)[,-1]
x[1:5,]
y <- Heart$AHD
g1 <- glmnet(x, y, family="binomial")
plot(g1, "lambda", label=TRUE)
```

Example: Heart Data

```
names(g1)
g1$df
g1$lambda
df <- cbind(g1$lambda, g1$df)
colnames(df) <- c("lambda", "nonzero")
rownames(df) <- colnames(g1$beta)
df
```

```
dim(g1$beta)
g1$beta[ , 1:5]
g1$beta[ , 65:69]
g1$beta[, c("s0", "s5", "s15", "s25", "s35")]
```

```
s35 <- as.numeric(coef(g1, s=g1$lambda[35]))
s20 <- as.numeric(coef(g1, s=g1$lambda[20]))
s10 <- as.numeric(coef(g1, s=g1$lambda[10]))
s02 <- as.numeric(coef(g1, s=g1$lambda[2]))
data.frame(LS=g0$coef, lasso.s35=s35, lasso.s20=s20,
           lasso.s10=s10, lasso.s02=s02)
```

Example: Heart Data

```
set.seed(12345)
gcv <- cv.glmnet(x, y, family="binomial", nfolds=5)
plot(gcv)
```

```
names(gcv)
data.frame(lambda=gcv$lambda, df=gcv$nzero, CVE=gcv$cvm)
```

```
gcv$lambda.min
min(gcv$cvm)
coef(gcv, s="lambda.min")
```

```
g1 <- glmnet(x, y, family="binomial")
plot(g1, "lambda", label=TRUE)
abline(v=log(gcv$lambda.min), col="red", lty=2)
```

Example: Heart Data

```
set.seed(111)
g1 <- glmnet(x, y, family="binomial")
plot(g1, "lambda", label=TRUE)
```

```
Lambda <- nzero <- NULL
for (i in 1:10) {
  gcv <- cv.glmnet(x, y, family="binomial", nfolds=5)
  abline(v=log(gcv$lambda.min), col="red", lty=2)
  Lambda[i] <- gcv$lambda.min
  cf <- coef(gcv, s="lambda.min")[-1]
  nzero[i] <- sum(cf!=0)
}
```

```
data.frame(opt.lambda=Lambda)
table(round(Lambda, 5))
nzero
```


Example: Heart Data

```
set.seed(111)
g01 <- cv.glmnet(x, y, family="binomial", type.measure="deviance")
```

```
set.seed(111)
g02 <- cv.glmnet(x, y, family="binomial", type.measure="class")
```

```
set.seed(111)
g03 <- cv.glmnet(x, y, family="binomial", type.measure="auc")
```

```
par(mfrow=c(1, 3))
plot(g01)
plot(g02)
plot(g03)
```

Example: Heart Data

```
lambda.min <- c(g01$lambda.min, g02$lambda.min, g03$lambda.min)
names(lambda.min) <- c("deviance", "class", "auc")
data.frame(lambda.min)
```

```
dev.off()
plot(g1, "lambda", label=TRUE)
abline(v=log(g01$lambda.min), col="red", lty=2)
abline(v=log(g02$lambda.min), col="red", lty=2)
abline(v=log(g03$lambda.min), col="red", lty=2)
```

```
cf01 <- coef(g01, s="lambda.min")
cf02 <- coef(g02, s="lambda.min")
cf03 <- coef(g03, s="lambda.min")
cf <- cbind(cf01, cf02, cf03)
colnames(cf) <- c("deviance", "class", "auc")
cf
```

1. Lasso for Variable Selection

2. Elastic-net for Gene Selection

3. Group Lasso for Genetic Pathway Selection

4. Analysis of High-dimensional DNA Methylation Data

Genetic Association Studies

- In genetic association studies, the goal is to identify SNP, genetic variants, genes or gene sets that are associated with diseases/traits.
- It is a statistically challenging problem due to
 - Problem of dimensionality, i.e., $p \gg n$
 - Highly correlated genomic data (LD)
 - Considering covariate effects (non-genetic effects)
 - Sparsity of true signals
- Univariate analysis where each variable (a group or variables) is tested one at a time cannot be accounted for correlation among genetic variables.
- Multiple linear regression cannot be applied because high variance of $\hat{\beta}$ and $p \gg n$.
- Alternatives: regularization methods (penalized regression)!

BIOINFORMATICS ORIGINAL PAPER

Vol. 27 no. 24 2011, pages 3399–3406
doi:10.1093/bioinformatics/btr591

Gene expression

Optimized application of penalized regression methods to diverse genomic data

Levi Waldron^{1,3}, Melania Pintilie², Ming-Sound Tsao³, Frances A. Shepherd³,
Curtis Huttenhower^{1,*} and Igor Jurisica^{3,*}

BIOINFORMATICS ORIGINAL PAPER

Vol. 25 no. 6 2009, pages 714–721
doi:10.1093/bioinformatics/btp041

Genome analysis

Genome-wide association analysis by lasso penalized logistic regression

Tong Tong Wu¹, Yi Fang Chen², Trevor Hastie^{2,3}, Eric Sobel⁴ and Kenneth Lange^{4,5,*}

BIOINFORMATICS ORIGINAL PAPER

Vol. 27 no. 19 2011, pages 2679–2685
doi:10.1093/bioinformatics/btr450

Systems biology

Advance Access publication July 30, 2011

Penalized regression elucidates aberration hotspots mediating subtype-specific transcriptional responses in breast cancer

Yinyin Yuan^{1,2,*}, Oscar M. Rueda^{1,2}, Christina Curtis^{1,2,†} and Florian Markowetz^{1,2,*}



RESEARCH ARTICLE

Open Access

Sparse logistic regression with a $L_{1/2}$ penalty for gene selection in cancer classification

Yong Liang^{1*}, Cheng Liu¹, Xin-Ze Luan¹, Kwong-Sak Leung², Tak-Ming Chan², Zong-Ben Xu³ and Hai Zhang³

Genome analysis

pETM: a penalized Exponential Tilt Model for analysis of correlated high-dimensional DNA methylation data

Hokeun Sun¹, Ya Wang², Yong Chen³, Yun Li^{4,5,6} and Shuang Wang^{2,*}

¹Department of Statistics, Pusan National University, Busan 609-735, Korea, ²Department of Biostatistics, Mailman School of Public Health, Columbia University, New York, NY 10032, USA, ³Division of Biostatistics, Perelman School of Medicine, University of Pennsylvania, Philadelphia, PA 19104, USA, ⁴Department of Biostatistics, ⁵Department of Genetics and ⁶Department of Computer Science, University of North Carolina, Chapel Hill, NC 27599, USA

*To whom correspondence should be addressed.
Associate Editor: John Hancock

Journal of Bioinformatics and Computational Biology
Vol. 16, No. 4 (2018) 1850010 (18 pages)
© World Scientific Publishing Europe Ltd.
DOI: 10.1142/S0219720018500105



Kim and Sun *BMC Bioinformatics* (2019) 20:510
<https://doi.org/10.1186/s12859-019-3040-x>

BMC Bioinformatics

New variable selection strategy for analysis
of high-dimensional DNA methylation data

METHODOLOGY ARTICLE

Open Access

Incorporating genetic networks into case-control association studies with high-dimensional DNA methylation data

Kipoong Kim and Hokeun Sun*

Jiyoung Choi, Kipoong Kim and Hokeun Sun*

Department of Statistics
Pusan National University
Busan 46241, Korea
*hsun@pusan.ac.kr



Regularization Methods

- **Regularization methods** are based on a penalized likelihood

$$Q_{\lambda}(\boldsymbol{\beta}) = -l(\boldsymbol{\beta}) + p_{\lambda}(\boldsymbol{\beta})$$

- $l(\boldsymbol{\beta})$ is a log-likelihood function and $\boldsymbol{\beta} = \{\beta_1, \dots, \beta_p\}$ is the p -dimensional vector of **regression coefficients**.
- $l(\boldsymbol{\beta})$ is determined by different **types** of phenotype outcomes.
- $p_{\lambda}(\boldsymbol{\beta})$ is a **penalty function**, where a **tuning parameter** λ controls sparsity.
- **Solution** to regression coefficient $\hat{\boldsymbol{\beta}}$ can be obtained by

$$\hat{\boldsymbol{\beta}}_{\lambda} = \arg \min_{\boldsymbol{\beta}} Q_{\lambda}(\boldsymbol{\beta}),$$

for a fixed value of λ .

Penalized Likelihood for Quantitative Outcome

- For the i -th individual,
 - y_i : a **quantitative** outcome
 - $x_i = (x_{i1}, \dots, x_{ip})^T$: the p -dimensional **genomic data**.
 - $z_i = (z_{i1}, \dots, z_{iq})^T$: the q -dimensional **covariates**.
- **Linear regression model** can be defined as

$$y_i = \beta_0 + z_i^T \gamma + x_i^T \beta + \epsilon_i,$$

where $\gamma = \{\gamma_1, \dots, \gamma_q\}^T$ and $\epsilon \sim N(0, \sigma^2)$.

- **Least squared loss** is used for the negative log-likelihood

$$\begin{aligned} Q_\lambda(\beta_0, \gamma, \beta) &= -l(\beta_0, \gamma, \beta) + p_\lambda(\beta) \\ &= \frac{1}{2n} \sum_{i=1}^n (y_i - \beta_0 - z_i^T \gamma - x_i^T \beta)^2 + p_\lambda(\beta) \end{aligned}$$

Penalized Likelihood for Binary Outcome

- For the i -th individual,
 - y_i : a **binary** outcome ($y_i = 1$ for cases and $y_i = 0$ for controls)
- In case-control studies, the penalized likelihood is

$$Q_\lambda(\beta_0, \gamma, \beta) = -l(\beta_0, \gamma, \beta) + p_\lambda(\beta).$$

- A **logistic likelihood** $l(\beta_0, \gamma, \beta)$ is

$$= \sum_{i=1}^n [y_i \log p_i(\beta_0, \gamma, \beta) + (1 - y_i) \log(1 - p_i(\beta_0, \gamma, \beta))],$$

where

$$p_i(\beta_0, \gamma, \beta) = \frac{e^{\beta_0 + z_i^T \gamma + x_i^T \beta}}{1 + e^{\beta_0 + z_i^T \gamma + x_i^T \beta}}.$$

Penalized Likelihood for Censored Survival Outcomes

- For the i -th individual,
 - t_i : Either survival or censoring time of the i -th individual
 - δ_i : the indicator of failure ($\delta_i = 1$) or censoring ($\delta_i = 0$)
- The penalized likelihood is

$$Q_\lambda(\gamma, \beta) = -\log L(\gamma, \beta) + p_\lambda(\beta).$$

- A Cox partial likelihood can be written as

$$L(\gamma, \beta) = \prod_{i=1}^n \left\{ \frac{\exp(z_i^T \gamma + x_i^T \beta)}{\sum_{j \in R_i} \exp(z_j^T \gamma + x_j^T \beta)} \right\}^{\delta_i},$$

where R_i is the index set for observations that are at risk before time t_i .

Types of Penalty Functions

■ Convex penalty functions

- Lasso (Tibshirani, JRSS, 1996)
- Fused lasso (Tibshirani *et al.* JRSS, 2005)
- Adaptive lasso (Zou, JASA, 2006)
- Elastic-net (Zou and Hastie, JRSS, 2005)

■ Non-convex penalty functions

- l_q -norm penalty with $0 < q < 1$
- Smoothly clipped absolute deviation (SCAD) (Fan and Li, JASA, 2005)
- Minimax concave penalty (MCP) (Zhang, AOS, 2010)

■ Group structure penalty functions

- Group lasso (Yuan and Lin, JRSS, 2006)
- Graph-constrained regularization (Li and Li, AOAS 2010)
- Sparse group lasso (Simon *et al.* JGCS 2013)
- Group exponential lasso (Breheny, Biometrics 2015)

Lasso and Ridge

- Lasso regression uses a l_1 -norm penalty

$$p_\lambda(\boldsymbol{\beta}) = \lambda \|\boldsymbol{\beta}\|_1 = \lambda \sum_{j=1}^p |\beta_j|$$

- Lasso can perform variable selection since

- $\hat{\beta}_j = 0$ for most of $j \in \{1, 2, \dots, p\}$.
- $\hat{\beta} \neq 0$ for only a few j .

- Ridge regression uses a squared l_2 -norm penalty

$$p_\lambda(\boldsymbol{\beta}) = \lambda \|\boldsymbol{\beta}\|_2^2 = \lambda \sum_{j=1}^p \beta_j^2$$

Lasso and Ridge

■ Variable selection

- Ridge regression **shrinks** the coefficient estimates towards **zero**, just like **lasso**.
- But, **ridge** regression coefficients are **not exactly equal to zero**, unlike **lasso**.
- Technically, **ridge** regression cannot perform **variable selection**.

■ Correlated variables

- Ridge regression is likely to **shrink** the coefficient estimates of **correlated variables** together.
- Lasso regression often selects only one of **correlated variables** even though all of them are related with a response.
- Lasso regression is not ideal for highly correlated predictors.

■ Neither **ridge** nor **lasso** will universally dominate the other.

Limitations of Lasso

- **Lasso** penalty function (l_1 -norm)
 - If $p > n$, **lasso** selects at most n variables. The number of selected genes is bounded by the number of samples.
 - **Lasso** is **indifferent** to highly correlated variables and tends to pick only one variable and ignore the rest.
- **Ridge** penalty function (squared l_2 -norm)
 - It cannot perform variable selection. (No coefficients are zero.)
 - But, ridge regression shrinks **correlated features** to each other
- **Elastic-net** regularization
 - The **elastic-net** combines the l_1 and squared l_2 penalties
 - The l_1 part of the penalty generates a **sparse** model.
 - The l_2 part of the penalty encourages **grouping effects**.

Elastic-net Regularization

- The penalty function of **elastic-net** is

$$\begin{aligned} p_{\lambda}(\boldsymbol{\beta}) &= \lambda \left(\alpha \|\boldsymbol{\beta}\|_1 + (1 - \alpha) \|\boldsymbol{\beta}\|_2^2 \right) \\ &= \lambda \alpha \sum_{j=1}^p |\beta_j| + \lambda (1 - \alpha) \sum_{j=1}^p \beta_j^2 \end{aligned}$$

where $\alpha \in [0, 1]$ is a **balance** between lasso and ridge.

- Lasso if $\alpha = 1$
- Ridge regression if $\alpha = 0$.
- Need to pick up the optimal λ and α .
 - α is often fixed as either 0.1 or 0.5.
 - λ is chosen by **cross-validation**.

Regularization with Covariates

- Suppose that we have a q -dimensional **covariate** vector. The covariate of the i -individual is

$$z_i = (z_{i1}, z_{i2}, \dots, z_{iq})^T$$

- The lasso **solution** $\hat{\beta}_\lambda$ minimizes the penalized likelihood

$$\frac{1}{n} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{k=1}^q \gamma_k z_{ik} - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p |\beta_j|,$$

if y_i is a **quantitative** outcome.

- Note that the regression coefficients of the **covariates** are not penalized.

One-standard-error Rule in Variable Selection

- Choose a smaller model: **one-standard-error rule**
- Calculate the standard error of the estimated test MSE for each λ .

$$sd(\text{CVE}(\lambda_l)) = \sqrt{\frac{1}{n-1} \sum_{k=1}^K \left(\text{MSE}_k(\lambda_l) - \frac{1}{n} \sum_{k=1}^K \text{MSE}_k(\lambda_l) \right)^2}$$

where

$$\text{MSE}_k(\lambda_l) = \sum_{i \in C_k} \left(y_i - x_i^T \hat{\beta}_{\lambda_l}^{[-k]} \right)^2$$

- Select the **smallest model** (**largest** λ) for which the test error is within **one standard error** of the lowest point on the curve.

$$[\min(\text{CVE}) - sd(\text{CVE}), \min(\text{CVE}) + sd(\text{CVE})]$$

Example: Breast Cancer Dataset

- An R package “`breastCancerMAINZ`” from the bioconductor contains the dataset “`mainz`”, which consists of 22,283 features with 200 samples.
 - `exprs(mainz)` : Matrix containing gene expressions as measured by Affymetrix hgu133a technology (single-channel, oligonucleotides).
 - `pData(mainz)` : AnnotatedDataFrame containing Clinical information of the breast cancer patients whose tumors were hybridized.
- We regarded a tumor size as the response variable.
- The main goal of the analysis is to identify genes associated with a tumor size.
- Reference
 - Schmidt et al. (2008) The humoral immune system has a key prognostic impact in node-negative breast. *Cancer Research* 68(13):5405-13.

Example: Breast Cancer Data

```
library(kogo)
```

```
data(BC)  
names(BC)  
str(BC)
```

```
dim(BC$x)  
BC$x[1:3, 1:10]  
head(BC$geneID, 10)  
BC$y
```

```
x <- BC$x  
y <- BC$y  
g1 <- glmnet(x, y)  
plot(g1, "lambda")
```

Example: Breast Cancer Data

```
BC$geneID[1:100, ]  
miss <- is.na(BC$geneID[,2])  
sum(miss)
```

```
gene.all <- BC$geneID[!miss, 2]  
length(gene.all)  
length(unique(gene.all))
```

```
x2 <- x[, !miss]  
colnames(x2) <- gene.all  
dim(x2)  
x2[1:3, 1:10]
```

```
z <- BC$z  
summary(z)  
z$age  
x3 <- cbind(z$age, x2)  
colnames(x3)[1] <- "age"
```

Example: Breast Cancer Data

```
dim(x3)
x3[1:3, 1:10]
```

```
gr <- glmnet(x3, y, alpha=0)
gl <- glmnet(x3, y, alpha=1)
```

```
par(mfrow=c(1,2))
plot(gr, "lambda")
plot(gl, "lambda")
```

```
pp <- rep(1, ncol(x3))
pp[1] <- 0
gp <- glmnet(x3, y, alpha=1, penalty.factor=pp)
```

```
dim(gp$beta)
gp$beta[1:4, 1:10]
data.frame(lambda=gp$lambda, df=gp$df)
```

Example: Breast Cancer Data

```
set.seed(123)
gcv <- cv.glmnet(x3, y, penalty.factor=pp, nfolds=5)
c(gcv$lambda.min, gcv$lambda.1se)
dev.off()
plot(gcv)
```

```
plot(gp, "lambda")
abline(v=log(gcv$lambda.min), lwd=2, lty=2, col="red")
abline(v=log(gcv$lambda.1se), lwd=2, lty=2, col="blue")
```

```
fit1 <- coef(gcv, s="lambda.min")[-1, ]
fit2 <- coef(gcv, s="lambda.1se")[-1, ]
c(sum(as.matrix(fit1)!=0), sum(as.matrix(fit2)!=0))
```

```
w1 <- which(as.matrix(fit1)!=0)
w2 <- which(as.matrix(fit2)!=0)
data.frame(coef=fit1[w1])
data.frame(coef=fit2[w2])
```

Example: Breast Cancer Data

```
set.seed(111)
g01 <- cv.glmnet(x3, y, alpha=0.1, penalty.factor=pp, nfolds=5)
g05 <- cv.glmnet(x3, y, alpha=0.5, penalty.factor=pp, nfolds=5)
```

```
par(mfrow=c(1, 2))
plot(g01)
plot(g05)
```

```
fit11 <- coef(g01, s="lambda.min")[-1, ]
fit12 <- coef(g01, s="lambda.1se")[-1, ]
c(sum(as.matrix(fit11)!=0), sum(as.matrix(fit12)!=0))
```

```
fit51 <- coef(g05, s="lambda.min")[-1, ]
fit52 <- coef(g05, s="lambda.1se")[-1, ]
c(sum(as.matrix(fit51)!=0), sum(as.matrix(fit52)!=0))
```

```
w5 <- which(as.matrix(fit51)!=0)
data.frame(genes=names(fit51[w5]), coef=fit51[w5])
```

Example: Breast Cancer Data

```
set.seed(1234)
foldid <- sample(rep(1:5, length=nrow(x3)))
table(foldid)
```

```
new.lam <- glmnet(x3, y, penalty.factor=pp)$lambda
alp <- seq(0.1, 1, 0.1)
cve <- df <- optlam <- NULL
for (j in 1:length(alp)) {
  g <- cv.glmnet(x3, y, alpha=alp[j], penalty.factor=pp,
                foldid=foldid, lambda=new.lam)
  cve[j] <- min(g$cvm)
  optlam[j] <- g$lambda.min
  df[j] <- g$nzzero[g$cvm==min(g$cvm)]
}
```

```
data.frame(alpha=alp, lambda=optlam, DF=df, min.CVE=cve)
dev.off()
plot(alp, cve, type="b", xlab="alpha", ylab="Minimum CVE")
```


1. Lasso for Variable Selection
2. Elastic-net for Gene Selection

3. Group Lasso for Genetic Pathway Selection

4. Analysis of High-dimensional DNA Methylation Data

High-dimensional Genomic Data with a Group Structure

- Group structured genomic data
 - **Single-nucleotide polymorphism (SNP)** data: genes or genetic regions includes multiple SNPs.
 - **DNA methylation** data: genes or genetic regions contains multiple CpG sites.
 - **Gene expression** data: genetic pathways or signaling pathways consist of multiple genes.
- Genomic measurements within the same group are likely to be **correlated** with each other.
- Penalty functions encouraging **grouping effects**
 - Group lasso (only group selection)
 - Group exponential lasso (both group and individual selection)
 - Sparse group lasso (both group and individual selection)
 - Network-based regularization (both group and individual selection)

- **Group lasso** performs group selection
- Suppose that the p predictors are divided into K **disjoint** groups of sizes q_k with corresponding coefficients β_k .
- **Group lasso** solves the convex optimization problem.

$$\hat{\beta} = \arg \min_{\beta} \left[-l(\beta) + \lambda \sum_{k=1}^K \sqrt{q_k} \|\beta_k\|_2 \right]$$

- $\|\beta_k\|_2 = 0$ if and only if $\beta_k = (0, \dots, 0)$, thus **entire groups are shrunk to 0**.
- Examples:
 - SNPs/variants/CpG sites in the same genes.
 - Genes grouped into functional pathways
 - Indicator variables for the levels of a categorical variable

BMC Bioinformatics



Research

Open Access

Identifying main effects and epistatic interactions from large-scale SNP data via adaptive group Lasso

Can Yang^{*1}, Xiang Wan¹, Qiang Yang², Hong Xue³ and Weichuan Yu^{*1}

Bioinformatics, 34(2), 2018, 278–285

doi: 10.1093/bioinformatics/btx594

Advance Access Publication Date: 18 September 2017

Original Paper

OXFORD

Bioimage informatics

A novel SCCA approach via truncated ℓ_1 -norm and truncated group lasso for brain imaging genetics

BMC Bioinformatics



Methodology article

Open Access

Supervised group Lasso with applications to microarray data analysis

Shuangge Ma^{*1}, Xiao Song² and Jian Huang³

Example: Breast Cancer Data

```
BC$geneID[1:100, ]  
miss <- is.na(BC$geneID[,2])  
sum(miss)
```

```
gene.all <- BC$geneID[!miss, 2]  
length(gene.all)  
length(unique(gene.all))
```

```
x2 <- x[, !miss]  
colnames(x2) <- gene.all  
dim(x2)
```

```
group.id <- as.numeric(factor(gene.all))  
oo <- order(group.id)  
head(data.frame(gene=gene.all, id=group.id)[oo, ], 50)  
tail(data.frame(gene=gene.all, id=group.id)[oo, ], 50)
```

Example: Breast Cancer Data

```
ss <- as.numeric(table(group.id))  
table(ss)
```

```
gr <- group.id[oo]  
gr[1:100]  
length(gr)  
max(gr)
```

```
sx <- x2[,oo]  
gg <- gglasso(sx, y, group=gr, dfmax=100)  
data.frame(lambda=gg$lambda, df=gg$df)  
plot(gg)
```

```
k <- 30  
abline(v=log(gg$lambda[k]), col="red", lty=2, lwd=1.5)
```

Example: Breast Cancer Data

```
dim(gg$beta)
bk <- gg$beta[,k]
as.matrix(bk[bk!=0], ncol=1)
```

```
sel.gene <- names(bk[bk!=0])
length(sel.gene)
table(sel.gene)
gg$beta[sel.gene, c("s0", "s5", "s15", "s20", "s25")]
```

```
##### Do not Run #####
# set.seed(123)
# ggcvcv <- cv.gglasso(sx, y, group=gr, dfmax=100, nfolds=5)
# plot(ggcvcv)
# ggcvcv$lambda.min
# ggcvcv$lambda.1se
#####
```

Overlapping Group Lasso

- If K groups are **not disjoint**, then **overlapping group lasso** can be applied.
- There are 4 **variables** such as x_1, x_2, x_3 and x_4 and 4 **groups** such as $\{x_1, x_2\}$, $\{x_2, x_3\}$, $\{x_1, x_3\}$ and $\{x_3, x_4\}$,

$$\begin{array}{c}
 X \quad \beta \quad \gamma^1 \quad \gamma^2 \quad \gamma^3 \quad \gamma^4 \quad \sum_j \gamma^j \\
 \begin{array}{|c|} \hline x_1 \\ \hline x_2 \\ \hline x_3 \\ \hline x_4 \\ \hline \end{array}
 \begin{array}{|c|} \hline \beta_1 \\ \hline \beta_2 \\ \hline \beta_3 \\ \hline \beta_4 \\ \hline \end{array}
 =
 \begin{array}{|c|} \hline \gamma_1^1 \\ \hline 0 \\ \hline \gamma_2^1 \\ \hline 0 \\ \hline \end{array}
 +
 \begin{array}{|c|} \hline 0 \\ \hline \gamma_2^2 \\ \hline \gamma_2^3 \\ \hline 0 \\ \hline \end{array}
 +
 \begin{array}{|c|} \hline 0 \\ \hline \gamma_1^3 \\ \hline 0 \\ \hline \gamma_3^3 \\ \hline 0 \\ \hline \end{array}
 +
 \begin{array}{|c|} \hline 0 \\ \hline 0 \\ \hline \gamma_3^4 \\ \hline \gamma_4^4 \\ \hline \end{array}
 =
 \begin{array}{|c|} \hline \gamma_1^1 \\ \hline \gamma_1^3 \\ \hline \gamma_2^1 \\ \hline \gamma_2^2 \\ \hline \gamma_2^3 \\ \hline \gamma_3^3 \\ \hline \gamma_3^4 \\ \hline \gamma_4^4 \\ \hline \end{array}
 \end{array}$$

$$X\beta = X \sum_{j=1}^4 \gamma^j = \tilde{X}\gamma,$$

where $\tilde{X} = (x_1, x_1, x_2, x_2, x_3, x_3, x_3, x_4)$.

Duplication of Overlapped Variables

- Suppose that the observed values of the i -individual at p variables by

$$\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{ip})^T$$

- If the l -th variable are overlapped by k_l different groups, we can **duplicate** the corresponding l -th variables k_l times, i.e.,

$$\tilde{\mathbf{x}}_i = (\overbrace{x_{i1}, \dots, x_{i1}}^{k_1}, \overbrace{x_{i2}, \dots, x_{i2}}^{k_2}, \dots, \overbrace{x_{ip}, \dots, x_{ip}}^{k_p})^T$$

- If the l -th variables belong to only one group, $k_l = 1$.
- The p -dimensional vector $\mathbf{x}_i$ is **converted** to the q -dimensional vector $\tilde{\mathbf{x}}_i$, where

$$q = \sum_{l=1}^p k_l.$$

Overlapping Group Lasso in Pathway Analysis

- **Overlapping group lasso** is a simple extension from **group lasso**, where overlapped variables are duplicated.
- It has been recently applied to a **genetic pathway selection** problem as an alternative method of gene set enrichment analysis (GSEA).

[Cancer Inform.](#) 2016; 15: 179–187.

PMCID: PMC5026200

Published online 2016 Sep 15. doi: [10.4137/CIN.S40043](#)

PMID: [27679461](#)

Overlapping Group Logistic Regression with Applications to Genetic Pathway Selection

[Yaohui Zeng](#) and [Patrick Breheny](#)

- In multiple genetic pathways, many genes are overlapped.
- Suppose that each pathway is regarded as a group and an individual gene is a member of group. Then, **overlapping group lasso** can identify genetic pathways associated with a phenotype outcome.

Example: Breast Cancer Data

```
gene.kegg <- BC$path.kegg  
gene.kegg[1:10]  
length(gene.kegg)  
lapply(gene.kegg, length)  
sz <- as.numeric(lapply(gene.kegg, length))  
sort(sz)
```

```
whh <- which(sz > 10 & sz < 80)  
sel.path <- gene.kegg[whh]  
length(sel.path)  
lapply(sel.path, length)
```

```
all.kegg <- unique(unlist(sel.path))  
length(all.kegg)  
ngene <- unique(colnames(sx))  
length(ngene)  
length(intersect(ngene, all.kegg))
```

Example: Breast Cancer Data

```
xp <- ig <- gid <- gname <- NULL
for(i in 1:length(sel.path)) {
  ig <- intersect(sel.path[[i]], ngene)
  k <- 0
  gsy <- NULL
  for (j in 1:length(ig)) {
    wh <- which(ig[j]==colnames(sx))
    xp <- cbind(xp, sx[,wh,drop=FALSE])
    k <- k + length(wh)
    gsy <- c(gsy, rep(ig[j], length(wh)))
  }
  gid <- c(gid, rep(i, k))
  gname <- c(gname, gsy)
}
colnames(xp) <- gname
```

```
dim(xp)
c(length(gid), length(gname))
c(length(unique(gname)), max(gid))
```

Example: Breast Cancer Data

```
ogl <- gglasso(xp, y, group=gid, dfmax=15)
data.frame(lambda=ogl$lambda, ogl$df)
```

```
k <- 22
plot(ogl)
abline(v=log(ogl$lambda[k]), col="red", lty=2, lwd=1.5)
```

```
wd <- unique(gid[ogl$beta[,k]!=0])
wd
```

```
path.name <- NULL
pathsel <- vector(mode="list", length=length(wd))
for (i in 1:length(wd)) {
  pathsel[[i]] <- sel.path[[wd[i]]]
  path.name <- c(path.name, names(sel.path[wd[i]]))
}
names(pathsel) <- path.name
pathsel
```

Bi-level Selection

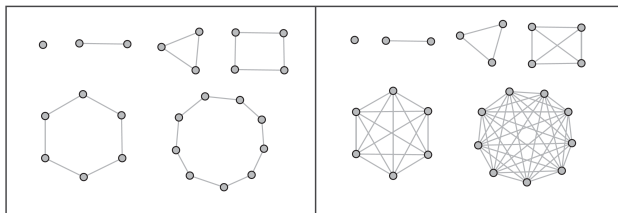
- Group lasso introduced the idea of **grouped variables** and the idea of selecting important groups of variables, rather than individual variables.
- There are often situations where we might be interested in selection at both the individual and group levels, or **bi-level selection**.
- **Grouping** made sense here: if a group is unimportant to the response, we don't want to select any members from it.
- However, **selecting individual members** also makes sense: just because a group is important to the response doesn't mean that every single member is important.
- Even if a group is important, a group can contain unimportant members. In this case, **group lasso** fails to identify unimportant members.

Bi-level Selection

- Sparse group lasso (SGL)
 - Additive penalty with **lasso** and **group lasso**
 - Computationally expensive
 - R package : “**SGL**”
- Group exponential lasso (GEL)
 - Multiplicative penalty with **lasso** and an **exponential** function
 - Computationally efficient
 - R package : “**grpreg**”
- Network-based regularization
 - Elastic-net penalty with a **Laplacian** matrix
 - It can be applied to either a **network-structured** data or a **group-structured** data
 - Computationally efficient
 - R package : “**pclogit**”

Network-based Regularization

- **Network-based regularization** was used for group structured data (Sun and Wang, 2012).
- Imaginary network structure for each gene
 - **Ring network**: adjacent sites/variants are only connected with each other
 - **Fully connected (F.con) network**: all sites/variants are fully connected with each other



Ring

F.con

- Laplacian matrix has a **block diagonal** matrix

Selection Probability

- **Stable selection** of regularization methods:
 - Selection probability (Meinshausen and Bühlmann, 2010)
 - Pairs stability selection (Shah and Samworth, 2013)
- Computation of **selection probability**:
 - Let I_k be the k -th bootstrap samples.
 - For a fine grid of λ , the selection probability of the j -th gene is

$$SP_j = \max_{\lambda} \frac{1}{K} \# \{k \leq K : \hat{\beta}_j^{\lambda}(I_k) \neq 0\},$$

where K is the total number of resampling.

- $\hat{\beta}_j^{\lambda}(I_k)$ is the estimated regression coefficient for the j -th gene based on the k -th bootstrap sample when λ is fixed.
- SP measures **relative selection frequency** of individual genes.
- **Top ranked genes** can be listed along with selection probabilities of individual genes.

Example: Breast Cancer Data

```
g01 <- grpreg(sx, y, group=gr, penalty="gel")  
g02 <- SGL(list(x=sx, y=y), index=gr, alpha=0.5)
```

```
par(mfrow=c(1,2))  
plot(g01)  
title("GEL")  
matplot(rev(log(g02$lambda)), t(g02$beta), type="l", lty=1,  
        col="red", xlab="log.lambda", ylab="Coefficients",  
        main="SGL")
```

```
set.seed(111)  
gcv <- cv.grpreg(sx, y, group=gr, penalty="gel")  
dev.off()  
plot(gcv)
```

```
gcv.min <- coef(gcv)  
as.matrix(gcv.min[gcv.min!=0], ncol=1)
```

Example: Breast Cancer Data

```
library(pclogit)
```

```
y2 <- BC$z$e.dmf5  
table(y2)  
gsize <- as.numeric(table(gr))  
gsize[1:50]  
gr[1:50]
```

```
gp <- pclogit(sx, y2, alpha=0.5, group=gsize)  
dev.off()  
matplot(log(gp$lambda), t(gp$beta), type="l", lty=1, col="red",  
         xlab="log.lambda", ylab="Coefficients", main="pclogit")
```

```
##### Do not Run #####  
# sp <- sel.pclogit(sx, y2, alpha=0.5, group=gsize, K=100)  
# sp$maxsel[1:10,]  
#####
```

1. Lasso for Variable Selection
2. Elastic-net for Gene Selection
3. Group Lasso for Genetic Pathway Selection
- 4. Analysis of High-dimensional DNA Methylation Data**

DNA Methylation

- **DNA methylation** is the addition of a methyl group to the 5' position of cytosine in the context of a CpG dinucleotide.
- Adding methyl groups change the appearance and structure of DNA.

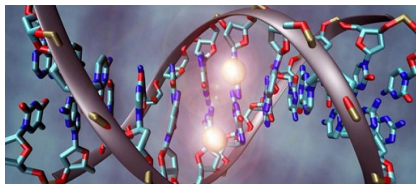
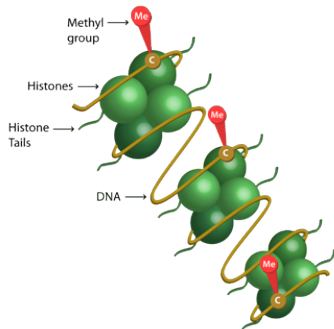


Illustration of a DNA molecule that is methylated at the cytosines.

High-dimensional DNA Methylation Data

- CpG (Cytosine-phosphate-Guanine) rich regions are frequently hyper-methylated in **cancer tissues**, but not in normal tissues.
- **DNA methylation data** have recently been generated from high-throughput DNA methylation platforms
 - Illumina Infinium HumanMethylation450 BeadChip array.
 - Illumina Infinium MethylationEPIC BeadChip array.

Design Markers | Process BeadChip | Analyze Data

illumina®

Infinium® MethylationEPIC BeadChip

Affordable methylome analysis meets cutting edge content.

Highlights

- **Unique Combination of Coding Region and Enhancer-Wide Coverage**
Over 850,000 methylation sites per sample at single-nucleotide resolution
- **High Assay Reproducibility**
> 98% reproducibility for technical replicates
- **Simple Workflow**
PCR-free protocol with the powerful Infinium HD Assay
- **Compatible with FFPE Samples**
Protocol available for methylation studies on FFPE samples



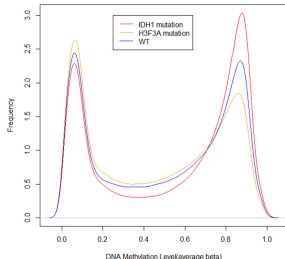
Methylation β -values

- The methylation array data of the CpG sites are a sequence of β -values.
- For each locus (CpG site)

$$\beta = \frac{\max(M, 0)}{\max(U, 0) + \max(M, 0) + 100},$$

where U is the fluorescent signal from an unmethylated allele and M is that from a methylated allele.

- This β -value ranges continuously from 0 (unmethylated) to 1 (completely methylated)
- The methylation levels are enriched at 0 and 1.



Analysis of High-dimensional DNA Methylation Data

■ Research goal

- Identify differentially methylated CpG sites and genes between cancer cases and healthy controls over entire genomes.
- (Matched) case-control association study with high-dimensional DNA data

■ Statistical problem

- Variable selection for high-dimensional data
- Methylation values of multiple CpG sites within the same genes are highly correlated with each other.

■ Regularization methods have been proposed.

- Sun and Wang (2012, *Bioinformatics*)
- Sun and Wang (2013, *Stat. Med.*)
- Sun *et al.* (2017, *Bioinformatics*)
- Choi *et al.* (2018, *J. Bioinform. Comput. Biol.*)
- Kim and Sun (2019, *BMC Bioinformatics*)
- Kim *et al.* (2020, *J. Stat. Comput. Simul.*)

Ovarian Cancer Study from 27K DNA Methylation Data

- The data is available at the NCBI Gene Expression Omnibus.
- Investigated differentially methylated CpG sites between ovarian cancer cases and healthy controls.
 - Teschendorff, A. *et al.* (2010, *Genome Research*)
 - Wang, S. (2011, *Genetic Epidemiology*)
 - Sun and Wang (2012, *Bioinformatics*)
 - Huang, H. *et al.* (2013, *BMC Bioinformatics*)
 - Chen, Y. *et al.* (2014, *Genetic Epidemiology*)
- Data from Illumina Infinium HumanMethylation27K Beadchip.
- After data processing, we ended up with 20,461 CpG sites from 12,770 genes.
- 12,770 genes have from 1 to 22 CpG sites.
- 152 healthy controls and 123 ovarian cancer cases.

Our Analysis for 27K DNA Methylation Data

- We identify differentially methylated CpG sites and the corresponding genes that associated with ovarian cancer.
- We apply 10 different ways (6 different statistical methods) to select cancer-related CpG sites
 - ① Univariate analysis (2) : Welch's two sample t-test and Wilcoxon-rank sum test with Bonferroni adjustment to control FWER (family-wise error rate) at 0.1.
 - ② Lasso (2) : CV with `lambda.min` and `lambda.1se`.
 - ③ Elastic-net (2) : CV with `lambda.min` and `lambda.1se`.
 - ④ Group lasso (2) : CV with `lambda.min` and `lambda.1se`.
 - ⑤ Group exponential lasso (1) : CV with `lambda.min`.
 - ⑥ Network-based regularization (1) : top 10 CpG sites ranked by their selection probability
- We finally summarize overlapping CpG sites and the corresponding genes.

Ovarian Cancer Data

```
data(ovarian27k)  
str(ovarian27k)
```

```
head(ovarian27k$genename)  
tail(ovarian27k$genename)
```

```
gsize <- ovarian27k$group  
c(length(gsize), sum(gsize))  
table(gsize)
```

```
ww <- which.max(gsize)  
cc <- cumsum(gsize)[(ww-1):ww]  
ovarian27k$genename[(cc[1]+1):cc[2], c(2,3)]
```

Ovarian Cancer Data

```
y <- ovarian27k$y  
y  
length(y)  
table(y)
```

```
x <- ovarian27k$x  
dim(x)  
summary(as.numeric(x))
```

```
xm <- log2(x/(1-x))  
summary(as.numeric(xm))
```

```
par(mfrow=c(1,2))  
hist(x, nclass=100, col="orange",  
     main=expression(paste(beta,"-values")))  
hist(xm, nclass=100, col="orange", main="M-values")
```

Univariate Analysis

```
pval.t <- apply(xm, 2, function(t) t.test(t~y)$p.val)
pval.w <- apply(xm, 2, function(t) wilcox.test(t~y)$p.val)
```

```
alpha <- 0.1
oo.t <- order(pval.t)
bt <- p.adjust(pval.t, method="bonferroni")
st <- sum(bt < alpha)
ot <- oo.t[1:st]
gt <- ovarian27k$genename[ot, c(2,3)]
data.frame(gt, p.value=pval.t[ot], Bonferroni=bt[ot])
```

```
oo.w <- order(pval.w)
bw <- p.adjust(pval.w, method="bonferroni")
sw <- sum(bw < alpha)
ow <- oo.w[1:sw]
gw <- ovarian27k$genename[ow, c(2,3)]
data.frame(gw, p.value=pval.w[ow], Bonferroni=bw[ow])
```

Univariate Analysis

```
par(mfrow=c(2, 1))
cc <- rep("gray", length(pval.t))
cc[bt < alpha] <- "darkblue"
plot(-log10(pval.t), type="p", pch=20, col=cc, xlab="CpG sites",
     ylab=expression(paste("-log"[10], "(P)")),
     main="Independent two sample T-test")
abline(h=-log10(max(pval.t[bt < alpha])), col="darkblue",
       lty=3, lwd=1.5)
```

```
cw <- rep("gray", length(pval.w))
cw[bw < alpha] <- "darkblue"
plot(-log10(pval.w), type="p", pch=20, col=cw, xlab="CpG sites",
     ylab=expression(paste("-log"[10], "(P)")),
     main="Wilcoxon Rank Sum Test")
abline(h=-log10(max(pval.w[bw < alpha])), col="darkblue",
       lty=3, lwd=1.5)
```

Lasso: Ovarian Cancer Data

```
g1 <- glmnet(xm, y, alpha=1, family="binomial")
dev.off()
plot(g1, "lambda")
g1$df
```

```
set.seed(1111)
gv1 <- cv.glmnet(xm, y, alpha=1, family="binomial", nfolds=5)
plot(gv1)
```

```
fit11 <- coef(gv1, s="lambda.min")[-1, ]
fit12 <- coef(gv1, s="lambda.1se")[-1, ]
c(sum(as.matrix(fit11)!=0), sum(as.matrix(fit12)!=0))
```

```
w11 <- which(as.matrix(fit11)!= 0)
w12 <- which(as.matrix(fit12)!= 0)
gn11 <- ovarian27k$genename[w11, c(2,3)]
gn12 <- ovarian27k$genename[w12, c(2,3)]
```

Lasso: Ovarian Cancer Data

```
data.frame(gn11, beta=fit11[w11])  
data.frame(gn12, beta=fit12[w12])
```

```
plot(g1, "lambda")  
abline(v=log(gv1$lambda.min), lty=2, col=2)  
abline(v=log(gv1$lambda.1se), lty=2, col=4)
```

```
par(mfrow=c(2,1))  
plot(abs(fit11), type="h", col="darkgreen", xlab="CpG sites",  
      ylab=expression(paste("|",hat(beta),"|")), ylim=c(0, 1.1),  
      main="Lasso using lambda.min")  
plot(abs(fit12), type="h", col="darkgreen", xlab="CpG sites",  
      ylab=expression(paste("|",hat(beta),"|")), ylim=c(0, 1.1),  
      main="Lasso using lambda.1se")
```


Elastic-net: Ovarian Cancer Data

```
g21 <- glmnet(xm, y, alpha=0.1, family="binomial")  
g22 <- glmnet(xm, y, alpha=0.5, family="binomial")
```

```
par(mfrow=c(1,2))  
plot(g21, "lambda")  
plot(g22, "lambda")
```

```
set.seed(1234)  
gv2 <- cv.glmnet(xm, y, alpha=0.5, family="binomial", nfolds=5)  
dev.off()  
plot(gv2)
```

```
fit21 <- coef(gv2, s="lambda.min")[-1, ]  
fit22 <- coef(gv2, s="lambda.1se")[-1, ]  
c(sum(as.matrix(fit21)!=0), sum(as.matrix(fit22)!=0))
```

```
w21 <- which(as.matrix(fit21)!= 0)  
w22 <- which(as.matrix(fit22)!= 0)
```

Elastic-net: Ovarian Cancer Data

```
gn21 <- ovarian27k$genename[w21, c(2,3)]  
gn22 <- ovarian27k$genename[w22, c(2,3)]  
data.frame(gn21, beta=fit21[w21])  
data.frame(gn22, beta=fit22[w22])
```

```
plot(g22, "lambda")  
abline(v=log(gv2$lambda.min), lty=2, col=2)  
abline(v=log(gv2$lambda.1se), lty=2, col=4)
```

```
par(mfrow=c(2,1))  
plot(abs(fit21), type="h", col="darkgreen", xlab="CpG sites",  
      ylab=expression(paste("|",hat(beta),"|")), ylim=c(0, 1.1),  
      main="Elastic-net using lambda.min")  
plot(abs(fit22), type="h", col="darkgreen", xlab="CpG sites",  
      ylab=expression(paste("|",hat(beta),"|")), ylim=c(0, 1.1),  
      main="Elastic-net using lambda.1se")
```

Elastic-net: Ovarian Cancer Data

```
set.seed(111)
foldid <- rep(1:5, length=length(y))
tapply(y, foldid, table)
new.lam <- glmnet(xm, y, family="binomial")$lambda
```

```
alp <- seq(0.1, 1, 0.1)
cve <- df <- NULL
for (j in 1:length(alp)) {
  gv2 <- cv.glmnet(xm, y, alpha=alp[j], family="binomial",
                  foldid=foldid, lambda=new.lam)
  cve[j] <- min(gv2$cvm)
  df[j] <- gv2$nzzero[gv2$cvm==min(gv2$cvm)]
}
```

```
data.frame(alpha=alp, DF=df, min.CVE=cve)
dev.off()
plot(alp, cve, type="b", xlab="alpha", ylab="Minimum CVE")
```

Group Lasso: Ovarian Cancer Data

```
## library(gglasso)
```

```
grp <- rep(1:length(gsize), gsize)
ry <- y
ry[y==0] <- -1
```

```
##### Do not run #####
# g3 <- gglasso(xm, ry, group=grp, loss="logit")
#####
```

```
ur1 <- url("https://github.com/statsun78/kogo/blob/main/
           g35.Rdata?raw=true")
load(ur1)
```

```
g3$df
plot(g3)
```

Group Lasso: Ovarian Cancer Data

```
##### Do not run #####  
# set.seed(1111)  
# gv3 <- cv.gglasso(xm, ry, group=grp, loss="logit", nfolds=5)  
#####
```

```
fit31 <- coef(gv3, s="lambda.min")[-1]  
fit32 <- coef(gv3, s="lambda.1se")[-1]  
c(sum(as.matrix(fit31)!=0), sum(as.matrix(fit32)!=0))
```

```
w31 <- which(as.matrix(fit31)!= 0)  
w32 <- which(as.matrix(fit32)!= 0)  
gn31 <- ovarian27k$genename[w31, c(2,3)]  
gn32 <- ovarian27k$genename[w32, c(2,3)]
```

```
data.frame(gn31, beta=fit31[w31])  
data.frame(gn32, beta=fit32[w32])
```

Group Lasso: Ovarian Cancer Data

```
plot(g3)
abline(v=log(gv3$lambda.min), lty=2, col=2)
abline(v=log(gv3$lambda.1se), lty=2, col=4)
```

```
par(mfrow=c(2,1))
plot(abs(fit31), type="h", col="darkgreen", xlab="CpG sites",
      ylab=expression(paste("|",hat(beta),"|")), ylim=c(0, 1.1),
      main="Group Lasso using lambda.min")
plot(abs(fit32), type="h", col="darkgreen", xlab="CpG sites",
      ylab=expression(paste("|",hat(beta),"|")), ylim=c(0, 1.1),
      main="Group Lasso using lambda.1se")
```

GEL: Ovarian Cancer Data

```
g4 <- grpreg(xm, y, group=grp, family="binomial", penalty="gel")
dev.off()
plot(g4)
```

```
set.seed(12)
gv4 <- cv.grpreg(xm, y, group=grp, family="binomial",
                 penalty="gel")
abline(v=gv4$lambda.min, lty=2, col=2)
plot(gv4)
```

```
fit41 <- coef(gv4)[-1]
w41 <- which(fit41!=0)
gn41 <- ovarian27k$genename[w41, c(2,3)]
data.frame(gn41, beta=fit41[w41])
```

```
plot(abs(fit41), type="h", col="darkgreen", xlab="CpG sites",
     ylab=expression(paste("|",hat(beta),"|")), ylim=c(0, 3),
     main="Group Exponential Lasso")
```

Network-based Regularization: Ovarian Cancer Data

```
library(pclogit)
```

```
g5 <- pclogit(xm, y, alpha=0.5, group=gsizes)
matplot(log(g5$lambda), t(g5$beta), type="l", lty=1, col="red",
        xlab=expression(log(lambda)), ylab="Coefficient",
        main="Network-based regularization")
data.frame(lambda=g5$lambda, df=g5$df)
```

```
##### Do not run #####
# set.seed(1212)
# gv5 <- sel.pclogit(xm, y, alpha=0.5, group=gsizes, K=200)
#####
```

```
top <- 10
w51 <- as.numeric(gv5$maxsel[1:top,1])
gn51 <- ovarian27k$genename[w51, c(2,3)]
data.frame(gn51, SP=gv5$maxsel[1:top, 2])
```


Summary: Ovarian Cancer Data

```
ooo <- c(ot, ow, w11, w12, w21, w22, w31, w32, w41, w51)
ov <- rep(0, length=nrow(ovarian27k$genename))
for (i in 1:length(ooo)) {
  oi <- ooo[i]
  ov[oi] <- ov[oi] + 1
}
```

```
length(ov)
table(ov)
```

```
wo <- which(ov > 1)
gn6 <- ovarian27k$genename[wo, c(2,3)]
ff <- data.frame(gn6, Freq=ov[wo])
ok <- order(ov[wo], decreasing=TRUE)
Final <- ff[ok, ]
rownames(Final) <- seq(1, nrow(Final))
Final
```